

## PART 1 / HOW THE WEBSITE WORKS

# A traffic signal, explained.

A practical guide to the interface, the underlying DFA, and the distinction between the model and its animation.

## The project in one sentence

SignalLab visualizes a deterministic finite automaton that validates traffic-light changes: **Red** → **Green** → **Yellow** → **Red**. Any repeated color or out-of-order change enters the non-accepting trap state.

## Read the interface from left to right

| Area                            | What it shows                                                                                                       |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 01 · Live intersection          | Cars, the monitored traffic signal, current state, next valid input, transition count and replay count.             |
| 02 · Build your sequence        | The original valid and invalid presets, custom input, start/stop, single-step, reset and playback speed.            |
| 03 · The automaton              | A live state diagram and the complete transition function. The current state and latest transition are highlighted. |
| 04 · Transition trace           | The latest 80 events, newest first. Start events and replay resets are labeled separately from actual transitions.  |
| Formal definition & model notes | The five-tuple, acceptance language, scenario-start convention and modeling limitations.                            |

## What to watch during a run

The signal, state diagram, transition table and trace all reflect the same current DFA state. On green, cars accelerate through the intersection. On red, they approach and queue behind the line. On yellow, approaching cars stop while cars already committed finish crossing.

**Presentation route:** explain the screen → run a valid cycle → inspect the theory → trigger a fault → summarize what makes this a DFA. Part 2 gives you the exact words to use.

PART 1 / WEBSITE CONTROLS

# Run, inspect, repeat.

Use automatic playback for the visual demonstration and single-step mode when explaining a particular transition.

| Control           | Behavior                                                                                                                                                |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scenarios         | Choose one of the eight original presets. Valid examples follow the color cycle; invalid examples deliberately break it.                                |
| Custom input      | Enter individual R, G or Y symbols separated by spaces or commas. Lowercase is accepted. Combined symbols such as RG and unknown symbols are rejected.  |
| Start simulation  | Start a new run from the selected sequence. After manual stop or rejection, this begins again; it does not resume an old run.                           |
| Stop simulation   | End automatic stepping, freeze traffic and display flashing amber. This is a demonstration control outside the DFA alphabet.                            |
| Advance one step  | The first click initializes the sequence in step mode. Each subsequent click consumes the next input. Cars remain still so the theory can be discussed. |
| Run automatically | Continue the current run from step mode.                                                                                                                |
| Reset simulation  | Return to idle at qR; clear the trace and counters and restore car positions.                                                                           |
| 0.5× / 1× / 2×    | Slow down or speed up the animation and signal dwell durations together. Changing speed restarts the current dwell timer.                               |

## The first symbol is a setup choice

For **R G Y R**, R selects the starting light. The consumed input word is **G Y R**. For **G R**, G selects qG and the input R demonstrates the illegal qG → qR change. This preserves the original local-scenario behavior.

A one-symbol sequence is a valid setup with no input transitions. Its replay counter may increase, but the transition count stays zero. Inputs cannot be edited during automatic playback or step mode; reset or stop first.

## PART 1 / THE FORMAL MODEL

# The DFA behind the scene.

$M = (Q, \Sigma, \delta, q_{\square}, F)$ . The machine is deterministic because every state-input pair has one defined destination.

| Component                            | Definition                                                           |
|--------------------------------------|----------------------------------------------------------------------|
| $Q$ · states                         | $\{qR, qG, qY, qD\}$                                                 |
| $\Sigma$ · alphabet                  | $\{R, G, Y\}$                                                        |
| $q_{\square}$ · formal initial state | $qR$                                                                 |
| $F$ · accepting states               | $\{qR, qG, qY\}$                                                     |
| $\delta$ · transition function       | $\delta : Q \times \Sigma \rightarrow Q$ , defined completely below. |

## The complete transition function

| State / input | R    | G    | Y    |
|---------------|------|------|------|
| $qR$          | $qD$ | $qG$ | $qD$ |
| $qG$          | $qD$ | $qD$ | $qY$ |
| $qY$          | $qR$ | $qD$ | $qD$ |
| $qD$          | $qD$ | $qD$ | $qD$ |

**Valid edges:**  $\delta(qR, G) = qG$ ;  $\delta(qG, Y) = qY$ ;  $\delta(qY, R) = qR$ . Every other input from a normal state goes to  $qD$ . The trap has a self-loop for each symbol.

## Why three states are accepting

Acceptance means the input read so far obeys the allowed ordering. A valid input word need not end at red or complete a full cycle. The double circles in the diagram indicate  $qR$ ,  $qG$  and  $qY$  are accepting;  $qD$  has only one circle.

The alphabet is made of signal-change symbols. Cars, seconds, playback speed, start, stop and reset are not members of  $\Sigma$ .

## PART 1 / WORKED EXAMPLES

# Follow the input word.

Always state the starting configuration before describing the consumed inputs.

## Example A: the valid preset R G Y R

| Action  | DFA result             | Meaning                |
|---------|------------------------|------------------------|
| Setup R | Start at $q_R$         | No symbol consumed yet |
| Read G  | $\delta(q_R, G) = q_G$ | Valid; green is lit    |
| Read Y  | $\delta(q_G, Y) = q_Y$ | Valid; yellow is lit   |
| Read R  | $\delta(q_Y, R) = q_R$ | Valid; red is lit      |

## Example B: the invalid preset R Y

Initialize  $q_R$ , then consume Y. The table says  $\delta(q_R, Y) = q_D$ , because green was skipped. The diagram highlights the invalid edge, the table highlights the  $q_R/Y$  cell, the trace records the rejection and traffic freezes. The demonstration halts at the first invalid transition.

## The trap remains a trap

Mathematically, any remaining R, G or Y inputs would keep the machine in  $q_D$ . The interface stops consuming them so the fault is easy to inspect. Pressing Reset or starting a new run creates a new execution; neither is an escape edge in the DFA.

## What language is accepted from $q_R$ ?

The accepted words are all finite prefixes of the repeating word **GYRGYRGYR....** A regular expression is  **$(GYR)^*(\epsilon \mid G \mid GY)$** . Here  $\epsilon$  denotes the empty word. Examples:  $\epsilon$ , G, GY, GYR and GYRG. Words R, Y, GG and GR are rejected from  $q_R$ .

The formal DFA accepts  $\epsilon$  because  $q_R$  is accepting. The UI requires at least a starting-light symbol to configure a scenario; an empty text field is therefore disabled. Scenarios starting at G or Y are local demonstrations, not formal runs beginning at  $q_R$ .

## PART 1 / TIMING &amp; IMPLEMENTATION

# One model. Several views.

The DFA decides validity; the surrounding simulation makes those decisions visible.

## How the implementation connects

**dfa.js** defines the transition table, scenarios, strict parser and run advancement. **App.jsx** holds the current run and drives the timer, diagram, table and trace. **traffic.js** updates car positions and safe following distances. **Intersection.jsx** draws the road and vehicles and animates them with `requestAnimationFrame`.

## Timing at normal speed

| Light  | Display duration | Visible traffic behavior                    |
|--------|------------------|---------------------------------------------|
| Red    | 4.2 seconds      | Approach the stop line and queue            |
| Green  | 4.2 seconds      | Accelerate and cross                        |
| Yellow | 1.9 seconds      | Brake before the line; committed cars clear |

These durations are visualization settings, not constraints enforced by the DFA. Vehicle motion uses elapsed time, acceleration and braking. The front bumper is the reference for the stop line; cars keep a minimum following gap. Vehicles leaving the scene return off-screen behind the queue.

## Automatic replay is a new demonstration

At the end of a valid scenario, the UI resets to its first symbol and labels a **Replay reset** event. This is outside  $\delta$ . If the ending and starting colors match, the duplicate dwell is skipped. The counter measures scenario replays, not necessarily completed physical traffic cycles.

## Scope and limitations

The model validates one monitored approach. It does not coordinate a complete intersection, sense vehicles, control pedestrians, enforce real-world timing or provide emergency preemption. A plain DFA has no clock. Timing constraints require an extension such as a timed automaton.

The signal output depends only on the current state, which is a Moore-style output mapping. DFA acceptance and the visual output are related layers, not interchangeable definitions. Flashing fault and manual-stop indicators are demonstration conventions.

## PART 2 / WHAT TO SAY

# Your opening & valid demo.

Suggested speaking time: 2-3 minutes. Text in quotes is ready to say; action notes tell you what to click.

## 1. Introduce the purpose

“This is SignalLab, a traffic-light simulator for Formal Languages and Automata Theory. It demonstrates how a deterministic finite automaton can validate a sequence of signal changes. Our rule is red to green, green to yellow, and yellow back to red.”

## 2. Orient the audience

“On the left is the animated intersection. Below it I can choose a scenario or enter my own sequence. On the right are the live state diagram and transition table. They stay visible while the animation runs, so we can connect each visible signal change with its formal transition.”

## 3. Start the valid example

**Action:** Select Full cycle and click Start simulation. Use 1× speed. Keep the diagram and table visible.

“This sequence is R, G, Y, R. The first R sets the starting light to red. The actual input symbols are G, Y and R. When G is read from qR, the transition function gives qG, so the green light turns on and the cars move. Then Y takes us to qY. Finally R takes us back to qR.”

## 4. Connect movement to the state

“The animation follows the state: cars queue on red, accelerate on green and brake on yellow. A car already past the stop line completes its crossing. These motion details help visualize the result; the DFA itself only validates the order of the signal changes.”

**Presenter tip:** If the sequence moves too fast, reset and use Advance one step. The first click selects the starting state; later clicks consume one input each. You can then explain the highlighted table cell at your own pace.

## PART 2 / WHAT TO SAY

# Explain the model & fault.

Suggested speaking time: 2-3 minutes. Use the invalid example to make determinism and the trap state concrete.

## 5. State the five-tuple

“Formally,  $M$  is the five-tuple  $Q, \sigma, \delta, q_0$  and  $F$ .  $Q$  contains  $q_R, q_G, q_Y$  and  $q_D$ .  $\sigma$  contains  $R, G$  and  $Y$ . The formal initial state is  $q_R$ . The accepting states are  $q_R, q_G$  and  $q_Y$ , because each represents a valid sequence so far.  $q_D$  is the only non-accepting state.”

## 6. Explain determinism

“The table defines exactly one destination for every state and input. For example,  $\delta$  of  $q_R$  and  $G$  equals  $q_G$ . There are no alternative branches for the same pair. The transition function is total because all twelve state-input combinations are defined, including the trap-state row.”

## 7. Trigger and explain rejection

**Action:** Click Reset, choose Skips green, then click Start simulation. Wait for  $R \rightarrow Y$  to be rejected.

“Now I am testing  $R$  followed by  $Y$ . Red is the starting light and  $Y$  is the input. This skips green, so  $\delta$  of  $q_R$  and  $Y$  is  $q_D$ . The trace records the invalid transition and the simulator halts.  $q_D$  is absorbing: any further  $R, G$  or  $Y$  would keep the machine in  $q_D$ .”

## 8. Close with the right limitation

“This is an educational model of signal-order validation, not a complete road controller. The timing, vehicle movement and manual controls belong to the simulation layer. The main FLAT idea is that a finite set of states and one deterministic transition rule are enough to recognize whether the sequence follows the allowed pattern.”

**Optional closing line:** “The project makes the abstract transition function visible: one input changes the DFA state, the highlighted edge, the table cell and the traffic signal together.”

## PART 2 / PRESENTATION QUESTIONS

# Be ready for the follow-up.

Short answers to the questions most likely to arise during a classroom explanation.

## Why is this a DFA rather than an NFA?

Every state-input pair has exactly one next state. There is no nondeterministic choice and no need for backtracking.

## Why does $R \rightarrow R$ fail?

In this project, symbols represent requested changes. A repeated color is deliberately treated as invalid. That is a modeling decision, not a universal traffic-law rule.

## Why is qG accepting if the cycle is unfinished?

Acceptance means the prefix read so far is valid. The language does not require a complete return to red.

## Does qD have self-loops?

Yes. R, G and Y all map qD back to qD. Only the three normal states reject repeated-color self-transitions.

## Can a sequence start at green?

The UI allows a local demonstration starting at qG. The formal DFA initial state is still qR. The first displayed symbol configures the demonstration.

## Is the timer part of the automaton?

No. It schedules visualization steps. A DFA alone has no elapsed-time variable; explicit timing constraints need a richer model.

## Does replay validate the boundary between runs?

No. Replay is an explicit reset to the scenario start and is labeled separately. Acceptance is evaluated within each demonstration.

## What should I demonstrate if time is short?

Run Full cycle, identify one valid table entry, then run Skips green and explain qD. Finish by stating that exactly one outcome exists for every state-input pair.

**Before presenting:** open the site, keep the page near the top, use a desktop-width window for side-by-side theory, and have this PDF available locally. Source of truth: the project's `dfa.js`, `traffic.js`, `App.jsx` and `Intersection.jsx`.